

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



TESTES COM PHPUNIT E PEST

Criação e configuração de testes no Laravel com as bibliotecas
PHPUnit e Pest .

Prof. Gill Velleda Gonzales
gillgonzales@ifsul.edu.br | [@gillgonzales](#) | [@g1ll](#)
CSTSI – IV Semestre – Desenvolvimento Back-End II
Junho - 2025

Testes no Laravel

Tópicos da Aula



- **Conceitos Gerais**

- Tipos de Testes
- O triplo AAA

- **PHPUnit**

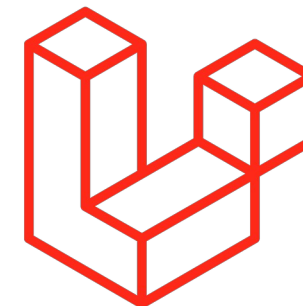
- Criação da classe de teste básica
- Exemplos e comandos do artisan

- **Pest**

- Configuração
- Exemplos de testes

- **Atividades**

- **Referências**

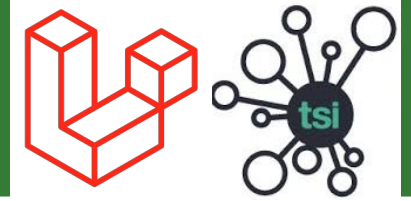


PHPUnit

PEST

Testes e Tipos para Back-end

Introdução aos diferentes tipos de testes



- Testes são realizados de diversas maneiras conforme o objetivo do teste. Podendo testar funcionalidades inteiras de uma aplicação (**End-to-End**), que envolve diversos componentes, módulos e etc. Ou situações específicas, isoladas, apenas comportamentos de um único módulo ou componente da interface **Unitário**.

- ▶ **Unitário (Unit):**

- Testa métodos isolados e seus comportamentos.



- ▶ **Funcionais (Feature):**

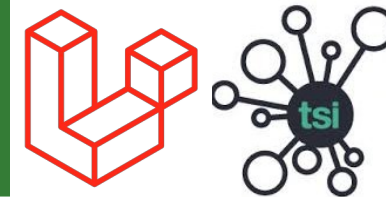
- Testa funcionalidades da aplicação, envolvendo várias camadas e simulando ações desempenhadas pelo sistema como um todo.

- ▶ **Outros:** existem outros tipos que focam em **integração** (APIs), **performance** (recursos de memória e tempo de carregamento), e ainda **acessibilidade** (Inclusão e diversidade de uso).



Principais Partes de um Teste

Quais são as partes principais que compoem um teste de software



- Para a realização de testes precisamos garantir algumas condições que envolvem o ambiente ou condições em que o software será testado. Normalmente em testes unitários aplicamos o **AAA**.

PEST

- ▶ **Arranjo:** (*describe, test, it*, etc...)

- **Configuração** do cenário de teste, descrição, mocks de dados, preparação de dependências.

- ▶ **Ação:** (a lógica a ser processada)

- **Execução** do comportamento testado, invocando métodos ou funções para uma determinada ação a ser testada.

PHPUnit

- ▶ **Asserção:** (*assert, expect, should*, etc..)

- **Afirmação** do resultado que deve estar de acordo com o esperado. Significa uma verificação **lógica** do que se espera com o código testado, podendo ser verdadeiro ou falso, em casos de falha no teste.

Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- É uma biblioteca de testes baseada em OOP, desenvolvida há mais tempo.
- O laravel traz o PHPUnit configurado, caso seja a opção escolhida quando criado o projeto com o Laravel new.
- Este também traz o arquivo de configuração **phpunit.xml** com as configurações de teste padrões do Laravel

```

Laravel

Which starter kit would you like to install?
None

Which testing framework do you prefer?
  ○ Pest
  > ● PHPUnit

```

TESTE...	phpunit.xml
composer.json	1 <?xml version="1.0" encoding="UTF-8"?
composer.lock	2 <phpunit xmlns:xsi="http://www.w3.org
package-lock.json	3 xsi:noNamespaceSchemaLocation
package.json	4 bootstrap="vendor/autoload.p
phpunit.xml	5 colors="true"
README.md	6 >
vite.config.js	7 <testsuites>

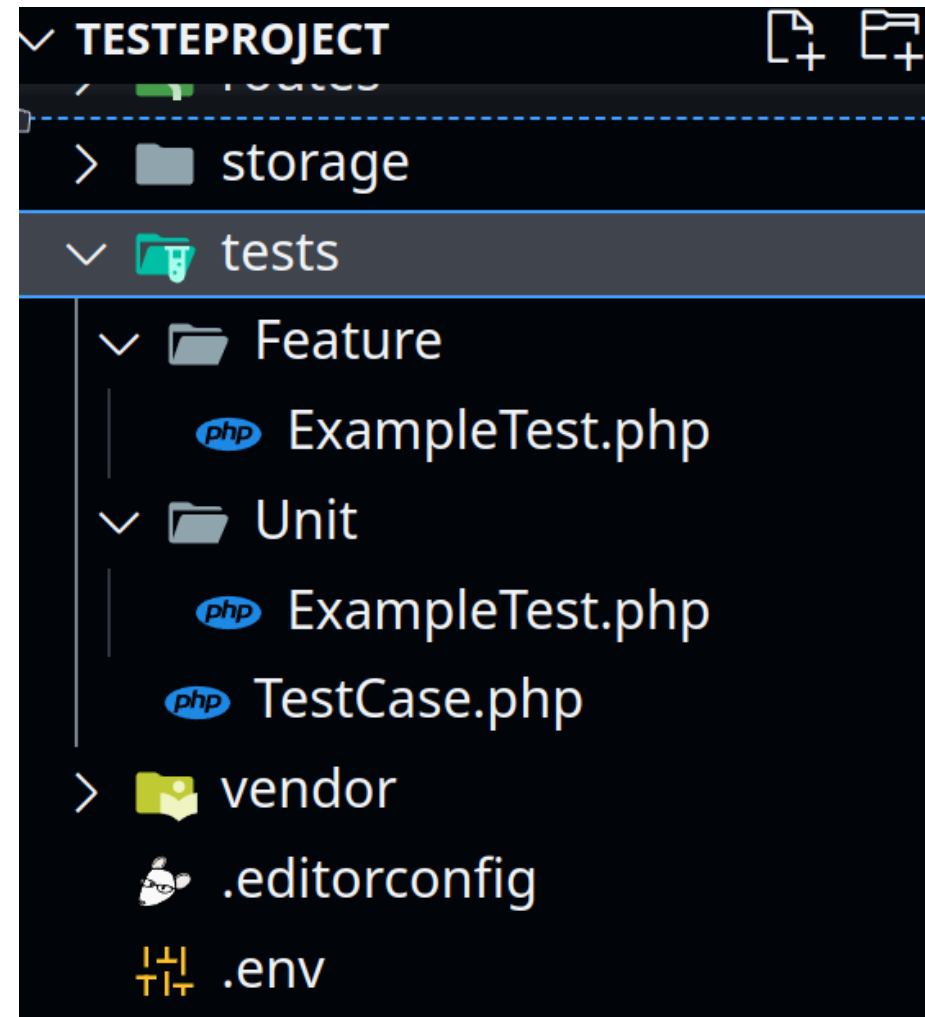
Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- O Laravel já traz como modelo e configuração dois tipos de testes:
 - **Feature (funcionalidade)**
 - Para testar funcionalidades inteiras, que envolvem outras partes do sistema, como uma autenticação por exemplo.
 - **Unit (unitário)**
 - Testa um método isolado, uma funcionalidade que não depende de outras partes do sistema, uma unidade.



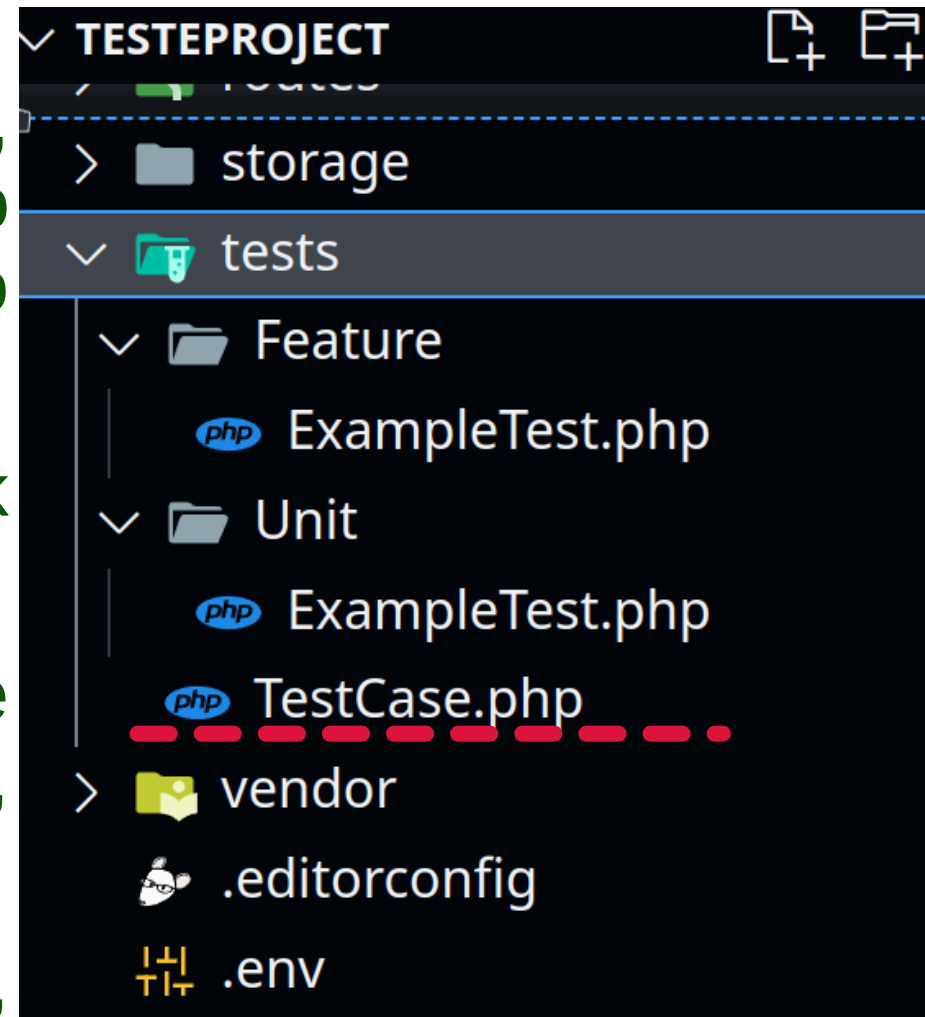
Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- **TestCase**: é a classe básica de testes, utilizada tanto pelo **PHPUnit**, quanto pelo **Pest**, ela estende a camada de testes do framework no *Illuminate\Foundation\Testing*.
- É a partir da classe básica que framework executa os seus testes.
- Por padrão o framework traz dois exemplos de testes, um na pasta Feature e outro na Unit, ambas classes chamadas **ExampleTest**.
- Para a classe ser reconhecida como teste, deve terminar com **Test**.



Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- **ExampleTest**: este é um exemplo de teste de funcionalidade (*Feature*), nele devemos criar o teste em um método iniciado pelo nome “test”, logo a descrição do teste se dá pelo nome do método no padrão *snake_case*.
- No caso do exemplo abaixo, a descrição do teste será “*the application returns a successful response*”.

tests > Feature > ExampleTest.php > ...

```
8  class ExampleTest extends TestCase
13      public function test_the_application_returns_a_successful_response(): void
14      {
15          $response = $this->get('/');
16
17          $response->assertStatus(200);
18      }
19  }
20
```


Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- Neste exemplo temos as fases de **execução (ACT)** e **asserção**, ou **afirmação lógica (ASSERT)**.
- O teste é considerado aprovado se a rota “/” retornar uma resposta de **status HTTP** igual a **200**.
- **assertStatus**: verifica o retorno da resposta HTTP e seu **status code**.

tests > Feature > ExampleTest.php > ...

```
8  class ExampleTest extends TestCase
13      public function test_the_application_returns_a_successful_response(): void
14      {
15          $response = $this->get('/');
16
17          $response->assertStatus(200);
18      }
19  }
```

Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- O teste unitário de exemplo faz uma verificação se o tipo true é true, usando o seu método **assertTrue**, herdado do PHPUnit Framework.

```
tests > Unit > ExampleTest.php > ...
5  use PHPUnit\Framework\TestCase;
6
7  class ExampleTest extends TestCase
8  {
9      /**
10       * A basic test example.
11       */
12     public function test_that_true_is_true(): void
13     {
14         $this->assertTrue(true);
15     }
16 }
```

- Nesta aula focaremos nos testes de funcionalidades (**Features**), por serem os mais frequentes.

Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- Para executar os testes, rodamos o comando do artisan test, com php ou com sail (docker).
- **PHP:** `$>php artisan test`
- **SAIL:** `$>sail artisan test`
- Ao executar o comando veremos a saída dos testes e a indicação se passaram ou falharam.

```
/media/NVME2T/Projetos/prototipos/testeProject
> php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response
```

Ferramentas de teste: PHPUnit

Configuração e utilização do PHPUnit no Laravel



Teste Com PHPUnit:

- Simulando um caso de falha.

```
tests > Unit > ExampleTest.php > ExampleTest > test_that_true_is_true
7  class ExampleTest extends TestCase
12      public function test_that_true_is_true(): void
13      {
14          $this->assertTrue(false);
15      }
16  }
```

PROBLEMS OUTPUT DEBUG CONSOLE DEVDB PORTS TERMINAL

🔍 /media/NVME2T/Projetos/prototipos/testeProject

```
> php artisan test
```

```
FAIL Tests\Unit\ExampleTest
✖ that true is true

PASS Tests\Feature\ExampleTest
✔ the application returns a successful response

PASS Tests\Feature\ExampleTestCopyTest
✔ copia the application returns a successful response

FAILED Tests\Unit\ExampleTest > that true is true
```

```
FAILED Tests\Unit\ExampleTest > that true is true
Failed asserting that false is true.

at tests/Unit/ExampleTest.php:14
10      * A basic test example.
11      */
12      public function test_that_true_is_true(): void
13      {
→ 14          $this->assertTrue(false);
15      }
16  }
17

1 tests/Unit/ExampleTest.php:14

Tests: 1 failed, 2 passed (3 assertions)
Duration: 0.16s
```


Testes no Banco de Teste

Configuração do banco de dados para testes



Banco de Dados de Teste:

- Para os testes não realizarem alterações no banco de dados padrão, podemos criar um banco para teste e especificar em um arquivo **ENV.TESTING**.
- Devemos criar o banco **testing** e executar as migrations indicando o **env de teste** e os **seeders**, se estiverem configurados.
- Mas antes de rodar as migrations, devemos preparar novos **seeders** e **factories** para Produto
- **Obs:** O banco de teste também deve ser preparado para os testes com o **PEST**.

```
.env.testing
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:S1CnnLWVYA1
4 APP_DEBUG=true
5
6 APP_LOCALE=en
7 APP_FALLBACK_LOCALE=pt_BR
8 APP_FAKER_LOCALE=pt_BR
9
10 DB_CONNECTION=mariadb
11 DB_HOST=mariadb
12 DB_PORT=3306
13 DB_DATABASE=testing
14 DB_USERNAME=sail
15 DB_PASSWORD=password
16
```



Testes no Banco de Teste

Configuração do banco de dados para testes



Criação do Banco de Dados de Teste:

- Para quem utiliza o *docker* com o *sail*, deverá entrar no serviço do banco de dados e **criar o banco testing** e de dar acesso (**grant**) para o usuário *sail*.

```
> sail exec mariadb bash
root@72fbd3ff5cca:/# mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 138
Server version: 11.7.2-MariaDB-ubu2404 mariadb.org binary distribution
MariaDB [(none)]> CREATE DATABASE testing;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON testing.* TO 'sail'@'%';
Query OK, 0 rows affected (0.007 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.004 sec)
```



Testes no Banco de Teste

Configuração do banco de dados para testes



Factory de Produto:

- Para popular o banco de teste, criaremos uma factory de produto, nos moldes da *UserFactory*.
- `$>php artisan make:factory ProdutoFactory -m Produto`
- `-m`: Indica a *model* ao qual a **factory** representa.
- Este comando criará a classe *ProdutoFactory* na pasta *database/factories*.

```
> sail artisan make:factory ProdutoFactory -m Produto
```

```
INFO Factory [database/factories/ProdutoFactory.php] created successfully.
```



Testes no Banco de Teste

Configuração do banco de dados para testes



Factory de Produto:

- Na classe gerada, alteramos o método **definition**, para retornar um array com os campos gerados aleatoriamente pelos métodos da biblioteca [fakerPhp](#), chamada via *helper* **fake()**.

```
CSTSI_202501_DBE2_PHP_LARAVEL  aula03 > database > factories > ProdutoFactory.php > ProdutoFactory
└─ aula03
   └─ database
      └─ factories
         └─ ProdutoFactory.php
            └─ UserFactory.php
               └─ migrations
                  └─ seeders
                     └─ DatabaseSeeder.php
                        └─ .gitignore
                           └─ public
                              └─ resources

10  class ProdutoFactory extends Factory
17
18      public function definition(): array
19      {
20          return [
21              "nome"=>fake()->sentence(5),
22              "descricao"=>fake()->sentence(20),
23              "preco"=>fake()->randomFloat(2, 10, 10000),
24              "qtd_estoque"=>fake()->randomNumber(2,10,10000),
25              "importado"=>fake()->numberBetween(0,1)
26          ];
27      }
```



Testes no Banco de Teste

Configuração do banco de dados para testes



Fazendo o Seeder de Produtos:

- Atualizamos a classe **DatabaseSeeder** para executar o método **create** da **factory** de produtos e criar a quantidade informada (20) de produtos aleatórios no banco de dados de teste.

A screenshot of an IDE interface. On the left, a file explorer shows a project structure with folders 'aula03', 'database', 'factories', 'migrations', and 'seeders'. The 'DatabaseSeeder.php' file is selected. On the right, the code editor shows the contents of 'DatabaseSeeder.php'. The code defines a 'DatabaseSeeder' class that extends 'Seeder' and implements a 'run()' method. Inside 'run()', it calls 'Produto::factory(20)->create()' to create 20 random products. The terminal at the top shows the command 'run' being executed.

```
aula03 > database > seeders > DatabaseSeeder.php > DatabaseSeeder > run
10  class DatabaseSeeder extends Seeder
15      public function run(): void
20          // ...
21          'email' => 'admin@test.dev',
22          'password'=> 'adminadmin'
23      });
24
25      Produto::factory(20)->create();
26  }
```

- Após todas essas configurações, factory de produto e atualização da seeder, rodamos as *migrations* no banco de teste:
- \$>php artisan migrate --env=testing --seed**



Criando um Teste com PHPUnit

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Vamos criar um teste para acessar a rota que lista todos os Produtos da nossa API, portanto, executaremos o seguinte comando do **artisan**:
 - **PHP:** `$>php artisan make:test Api/PHPUnit/ListProductTest`
 - **SAIL:** `$>sail artisan make:test Api/PHPUnit/ListProductTest`
- Serão criadas as pastas **Api** com **PHPUnit** e, dentro, a classe de teste **ListeProductTest.php**, tudo no diretório **tests/Feature**.

```
> php artisan make:test Api/PHPUnit/ListProductTest
```

```
INFO Test [tests/Feature/Api/PHPUnit/ListProductTest.php] created successfully.
```

- Não é necessário criar toda essa estrutura de diretórios, o **make:test** **sempre** criará o arquivo de teste na pasta **tests/Feature**, a não ser que o parâmetro **--unit** seja passado, indicando a criação de um teste **unitário** na pasta **tests/Unit**.

Criando um Teste com PHPUnit

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- A primeira classe de testes com todas as fases do triplo AAA.
 - Arranja (**ARRANGE**): prepara a url a ser acessada para o teste.
 - Atua (**ACT**): realiza a requisição do tipo get na rota alvo.
 - Asserta (**ASSERT**): afirma que o ***codigo HTTP*** será **200**.

tests > Feature > Api > PHPUnit > ListProductsTest.php > ListProductsTest > test_listar_todos_os_produtos

```
10 class ListProductsTest extends TestCase
15     public function test_listar_todos_os_produtos(): void
16     {
17         $URL = '/api/produtos'; //ARRANGE
18         $response = $this->get($URL); //ACT
19         $response->assertStatus(200); //ASSERT
20     }
21 }
```


Criando um Teste com PHPUnit

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Podemos executar todos os testes, ou apenas a nossa nova classe ***ListProductsTest***, passando o parâmetro ***--filter*** com o nome da classe entre chaves ***{nomeDaClasseTeste}***.

```
$>php artisan test --filter {ListProductTest}
```

🔍 ~/Projetos/prototipos/auth/csts-dbe2-202402-laravel-app on git 📁 Testing

```
• > sail artisan test --filter {ListProductsTest}
```

```
PASS Tests\Feature\Api\PHPUnit>ListProductsTest
✓ listar todos os produtos
```

```
Tests: 1 passed (1 assertions)
```

```
Duration: 0.38s
```

Criando Testes

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Considerando que o conteúdo da resposta seja um array JSON, poderíamos adicionar mais uma asserção em nosso teste.
- A função ***asserJsonIsArray()***, fará essa verificação no conteúdo do retornado no formato JSON.

```
public function test_listar_todos_os_produtos(): void
{
    $response = $this->get($URL); //ACT
    //ASSERT
    $response->assertStatus(200);
    $response->assertJsonIsArray();
}
```

A screenshot of a web browser showing the API response for the endpoint `http://localhost:8083/api/produtos/`. The browser's developer tools are open, showing the 'Test Results' tab with a status of '200 OK'. The 'Body' tab is selected, displaying a JSON array of product objects. A red dashed box highlights a portion of the JSON array, showing the structure of the objects. The JSON is as follows:

```
[
  {
    "id": 1,
    "nome": "Produto 1",
    "preco": 100,
    "fornecedor_id": 1
  },
  {
    "id": 2,
    "nome": "Produto 2",
    "preco": 200,
    "fornecedor_id": 2
  },
  {
    "id": 3,
    "nome": "Produto 3",
    "preco": 300,
    "fornecedor_id": 3
  },
  {
    "id": 4,
    "nome": "Produto 4",
    "preco": 400,
    "fornecedor_id": 4
  },
  {
    "id": 5,
    "nome": "Produto 5",
    "preco": 500,
    "fornecedor_id": 5
  }
]
```

Criando Testes

Criação de testes com PHPUnit no Laravel

Teste da Rota `/api/produtos` da API Com PHPUnit:

- Agora simulamos um erro, alterando a URL para uma rota que sabemos que não existe, retornando 404 e fazendo com que o teste Falhe **FAIL**.
- Neste exemplo fazemos apenas uma asserção, mas é possível realizar várias verificações em um mesmo teste adicionando outras asserções.

```
10 class ListProductsTest extends TestCase
15     public function test_listar_todos_os
17     {
18         $URL = '/api/produto'; //ARRANGE
19         $response = $this->get($URL); //ACT
20         $response->assertStatus(200); //ASSERT
21     }
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE DEVDB PORTS TERMINAL

~/Projetos/prototipos/auth/csts-dbe2-202402-laravel-app on git

```
> sail artisan test --filter {ListProductsTest}
```

FAIL Tests\Feature\Api\PHPUnit>ListProductsTest
x listar todos os produtos

FAILED Tests\Feature\Api\PHPUnit>ListProductsTest > listar todos os produtos
Expected response status code [200] but received 404.
Failed asserting that 404 is identical to 200.

at tests/Feature/Api/PHPUnit/ListProductsTest.php:19

```
15     public function test_listar_todos_os_produtos(): void
16     {
17         $URL = '/api/produto'; //ARRANGE
18         $response = $this->get($URL); //ACT
19         $response->assertStatus(200); //ASSERT
20     }
21 }
22 }
```

Tests: 1 failed (1 assertions)
Duration: 0.26s

Criando Testes

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Ao executar o teste verificamos que o código **PASSOU** no teste com **duas asserções**.
- No entanto, se pelo menos uma asserção não for atendida, **TODO** o teste **FALHA**.
- Existem diversos tipos de funções de asserção. Para verificar as diversas funções disponíveis para testes HTTP, visite a [Documentação](#).

```
public function test_listar_todos_os_produtos(): void
{
    $response = $this->get($URL); //API
    //ASSERT
    $response->assertStatus(200);
    $response->assertJsonisArray();
}
```

PASS Tests\Feature\Api\PHPUnit>ListProductsTest
✓ listar todos os produtos

Tests: **1 passed** (2 assertions)
Duration: 0.70s

```
public function test_listar_todos_os_produtos(): void
{
    $response = $this->get($URL); //API
    //ASSERT
    $response->assertStatus(200);
    $response->assertJsonIsObject();
}
```

FAIL Tests\Feature\Api\PHPUnit>ListProductsTest
× listar todos os produtos

FAILED Tests\Feature\Api\PHPUnit>ListProductsTest >
Failed asserting that false is true.

Teste da Rota `/api/produtos` da API Com PHPUnit:

- Alterando a **ResourceCollection** de Produto, a resposta HTTP deverá ser um **objeto JSON**, assim deverá passar no teste anterior.

```
app > Http > Resources > ProdutoCollection.php > ProdutoCollection > toArray
9  class ProdutoCollection extends ResourceCollection
16  public function toArray(Request $request): array
18  {
19      //JsonResource::withoutWrapping();
20      return parent::toArray($request);
21  }
```

- O método `withoutWrapping()`, remove o campo **data** do JSON retornado.
- Ao comentar esse método, recebemos novamente, um objeto `{data: [...]}`.

```
tests > Feature > Api > PHPUnit > ListProductsTest.php > ListProductsTest
10  class ListProductsTest extends TestCase
15  {
16      public function test_listar_todos_os_produtos(): void
17      {
18          $URL = '/api/produtos'; //ARRANGE
19          $response = $this->get($URL); //ACT
20          //ASSERT
21          $response->assertStatus(200);
22          $response->assertJsonIsObject();
23      }
24  }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  DEVDB  PORTS  TERMINAL
~/Projetos/prototipos/auth/csts-dbe2-202402-lar
> sail artisan test --filter {ListProductsTest}

PASS Tests\Feature\Api\PHPUnit\ListProductsTest
✓ listar todos os produtos

Tests: 1 passed (2 assertions)
Duration: 0.84s
```


Criando Testes

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Podemos avançar ainda mais nas asserções e verificar também a **estrutura do JSON** retornado com a função **`assertJsonStructure`**.
- Passamos um array associativo descrevendo a estrutura esperada do resultado **JSON** da api.
- Para outras funções de asserção em requisições HTTP, verifique a documentação.
- Funções de Asserção Disponíveis**

```
tests > Feature > Api > PHPUnit > ListProductsTest.php > ListProductsTest > t
10 class ListProductsTest extends TestCase
15     public function test_listar_todos_os_produtos(): void
16     {
17         $URL = '/api/produtos'; //ARRANGE
18         $response = $this->get($URL); //ACT
19         //ASSERT
20         $response->assertStatus(200);
21         $response->assertJsonIsObject();
22         $response->assertJsonStructure(["data"]);
23     }
24 }
25
```

PROBLEMS OUTPUT DEBUG CONSOLE DEVDB PORTS **TERMINAL**

```
~/Projetos/prototipos/auth/csts-dbe2-202402-laravel
> sail artisan test --filter {ListProductsTest}

PASS Tests\Feature\Api\PHPUnit\ListProductsTest
✓ listar todos os produtos

Tests: 1 passed (3 assertions)
Duration: 0.69s
```

Criando Testes

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com PHPUnit:

- Mas se queremos ainda mais detalhes, podemos verificar os tipos retornados na estrutura do objeto JSON, usando a classe ***AssertableJson***.
- Diversas queries podem ser montadas através desta classe.
- Com ***whereAllType***, verificamos os tipos no ***item 0*** do ***array*** no atributo ***data*** (***'data.0.id'***).
- ***Funções de Asserção***

```
tests > Feature > Api > PHPUnit > ListProductsTest.php > ListProductsTest > test_listar_todos_os_produtos

10 class ListProductsTest extends TestCase
15     public function test_listar_todos_os_produtos(): void
19         //ASSERT
20         $response->assertStatus(200);
21         $response->assertJsonIsObject();
22         $response->assertJsonStructure(["data"])
23         ->assertJson(fn(AssertableJson $json) => $json
24             ->whereAllType([
25                 'data.0.id' => 'integer',
26                 'data.0.nome' => 'string'
27             ]));
28     }
29 }
```

PROBLEMS OUTPUT DEBUG CONSOLE DEVDB PORTS TERMINAL

```
~/Projetos/prototipos/auth/csts-dbe2-202402-laravel-app on git Test:
> sail artisan test --filter {ListProductsTest}

PASS Tests\Feature\Api\PHPUnit\ListProductsTest
✓ listar todos os produtos

Tests: 1 passed (8 assertions)
Duration: 0.67s
```

Testes com a Pest

Criação de testes com Pest no Laravel



Ferramenta Pest:

- A Pest é um framework de testes alternativo ao conhecido PHPUnit.
- Usa uma abordagem mais simplificada, baseada em funções e não classes.
- O Laravel usa a sua estrutura para dividir testes unitários de testes funcionais.
- Sua instalação é muito simples, via composer ou através da escolha na instalação do laravel com o *laravel new*.
- Composer:

```
$>composer require pestphp/pest --dev
```

- Após a instalação, o arquivo *Pest.php* será criado na pasta **tests**.
- **Documentação**

Criando Testes

Criação de testes com PHPUnit no Laravel



Teste da Rota `/api/produtos` da API Com Pest:

- A grande mudança é que nossos testes não precisam mais serem escritos no formato de classes, mas sim apenas usando as funções do *framework* Pest.
- No exemplo ao lado, a função **`test()`** executa o nosso teste, recebendo uma **string** com a descrição do mesmo, e como segundo parâmetro a **função anônima** com o código de teste.

tests > Feature > Api > Pest > ListProductsPestTest.php > Closure

```
1  <?php
2
3  use Illuminate\Testing\Fluent\AssertableJson;
4  test('PEST: Listar todos os produtos', function () {
5      $URL = '/api/produtos'; //ARRANGE
6      $response = $this->get($URL); //ACT
7      //ASSERTAS
8      $response->assertStatus(200);
9      $response->assertJsonIsObject();
10     $response->assertStatus(200)
11     ->assertJson(fn (AssertableJson $json) =>$json
12     ->whereAllType([
13         'data.0.id' => 'integer',
14         'data.0.nome' => 'string'
15     ]));
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

DEVDB

PORTS

TERMINAL

~/Projetos/prototipos/auth/csts-dbe2-202402-laravel-app

```
> sail artisan test --filter {ListProductsPestTest}
```

PASS Tests\Feature\Api\Pest>ListProductsPestTest
✓ PEST: Listar todos os produtos

Tests: 1 passed (8 assertions)

Duration: 0.67s

Referências

Livros e documentações consultados



DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009.

ANICHE, Mauricio. **Testes automatizados de software: um guia prático**. São Paulo, SP: Casa do Código, 2015. E-book. Disponível em: <https://plataforma.bvirtual.com.br/Acervo/Publicacao/212643>. Acesso em: 04 jun. 2025.

Testing: Getting Started - Laravel 12.x - The PHP Framework For Web Artisans. disponível em: <https://laravel.com/docs/master/testing>, acesso em 03/06/2025.

HTTP Tests - Laravel Upcoming - The PHP Framework For Web Artisans, disponível em: <https://laravel.com/docs/master/http-tests>, acesso em 03/06/2025.

Database Testing - Laravel Upcoming - The PHP Framework For Web Artisans, disponível em: <https://laravel.com/docs/master/database-testing>, acesso em 03/06/2025.

Writing Tests | Pest - The elegant PHP Testing Framework. Disponível em: <https://pestphp.com/docs/writing-tests>. Acesso em: 03/06/2025.

PHPUnit Manual — PHPUnit 12.1 Manual. Disponível em: <https://docs.phpunit.de/en/12.1/index.html>. Acesso em: 03/06/2025.